Contact Manager System – Project Documentation

1. Introduction:

Project Title: Contact Manager System

Team Leader:Mathumitha A - mathumithankr@gmail.com

Team Members:

1.Hemalatha V - nkrhema703@gmail.com

2.Dhanalakshmi P - cithudhana@gmail.com

3.Dhanyasri S - sakhiyas310@gmail.com

2. Project Overview:

Purpose:

Contact Manager System is a web application designed to store and manage personal or professional contacts. It allows users to add, update, delete, and view contact information easily. This system helps users organize contact details efficiently and access them anytime.

Features:

- Add new contacts
- Update contact details
- Delete contacts
- View contact list
- Search contacts
- User login system

3. Architecture:

Frontend: HTML, CSS, JavaScript / React.js

Backend: Node.js with Express.js

Database: MongoDB

4. Setup Instructions:

Prerequisites:

Node.js

MongoDB

Git

Visual Studio Code

Installation Steps:

Copy code

```
git clone <repository-link>

cd client
npm install

cd ../server
npm install
```

5. Folder Structure:

Copy code

```
ContactManager/

|-- client/
|   |-- components/
|   |-- pages/

|-- server/
|   |-- routes/
|   |-- models/
|   |-- controllers/
```

6. Running the Application:

Frontend:

Copy code

cd client

npm start

Backend:

Copy code

cd server

npm start

Access:

http://localhost:3000

7. API Documentation:

User APIs

POST /api/user/register

POST /api/user/login

Contact APIs

GET /api/contacts

POST /api/contacts/add

PUT /api/contacts/update/:id

DELETE /api/contacts/delete/:id

8. Authentication:

JWT (JSON Web Token) is used for secure login and to protect private routes.

9. User Interface:

Login Page

Dashboard

Add Contact Page

Contact List Page

Update Contact Page

10. Testing:

Manual testing during development.

Tools Used:

Postman

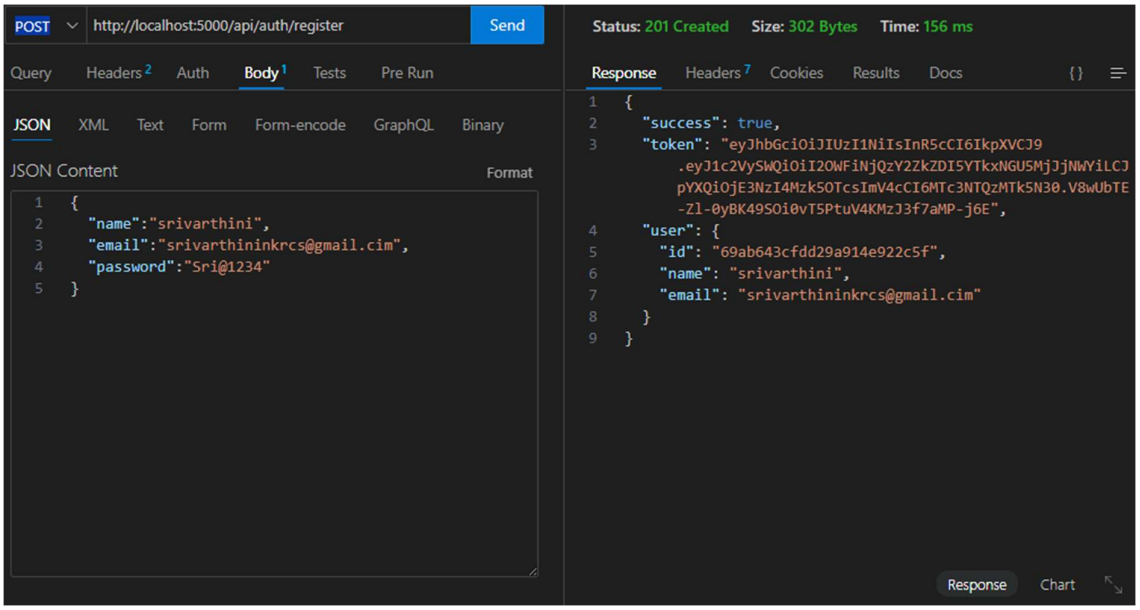
Chrome DevTools

Jest

11. Screenshots or Demo:

Add screenshots of:

1.Register Api



2.Login Api

POST http://localhost:5000/api/auth/login Send

Query Headers 3 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "email": "srivarthininkrcs@gmail.com",
  "password":"Sri@1234"
}
```

Status: 200 OK Size: 305 Bytes Time: 205 ms

Response Headers 7 Cookies Results Docs { }


```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJ1c2VySWQiOiI2OWFhOTNlMGMyYzRhOWU4MDNlMmE4ZDEiLCJ
      pYXQiOjE3NzI4Mzk1NjQsImV4cCI6MTc3NTQzMTU2NH0
      .e7hFKpUaM0E2OlHCWQFTuTLmt5RDWMBYq76qWNHtreg",
  "user": {
    "id": "69aa93e0c2c4a9e803e2a8d1",
    "name": "Sri varthini R",
    "email": "srivarthininkrcs@gmail.com"
  }
}
```


Response Chart

3.Create New Contact

POST http://localhost:5000/api/contacts Send

Query Headers 3 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "name": "sri varthini",
  "email": "srivarthininkrcs@gmail.com"
}
```

Status: 201 Created Size: 252 Bytes Time: 108 ms

Response Headers 7 Cookies Results Docs { }


```
{
  "success": true,
  "contact": {
    "userId": "69aa93e0c2c4a9e803e2a8d1",
    "name": "sri varthini",
    "email": "srivarthininkrcs@gmail.com",
    "tags": [],
    "_id": "69ab644dfdd29a914e922c62",
    "createdAt": "2026-03-06T23:33:33.505Z",
    "updatedAt": "2026-03-06T23:33:33.505Z",
    "__v": 0
  }
}
```


Response Chart

4.Update Contact

PUT http://localhost:5000/api/contacts/69ab644dfdd29a914e92 Send

Query Headers 3 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "phone":"6547892345",
  "tags":["test"]
}
```

Status: 200 OK Size: 279 Bytes Time: 58 ms

Response Headers 7 Cookies Results Docs { }


```
{
  "success": true,
  "contact": {
    "_id": "69ab644dfdd29a914e922c62",
    "userId": "69aa93e0c2c4a9e803e2a8d1",
    "name": "sri varthini",
    "email": "srivarthininkrcs@gmail.com",
    "tags": [
      "test"
    ],
    "createdAt": "2026-03-06T23:33:33.505Z",
    "updatedAt": "2026-03-06T23:52:15.561Z",
    "__v": 1,
    "phone": "6547892345"
  }
}
```


Response Chart

5.Delete Contact

PUT http://localhost:5000/api/contacts/69ab644dfdd29a914e92 Send

Query Headers 3 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "phone":"6547892345",
  "tags":["test"]
}
```

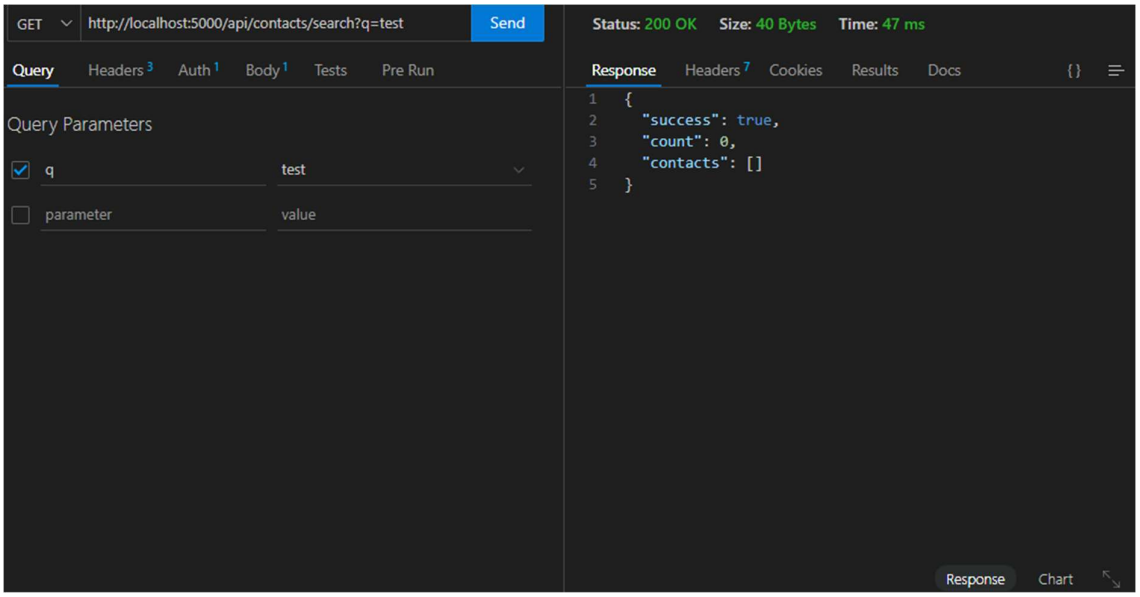
Status: 200 OK Size: 279 Bytes Time: 58 ms

Response Headers 7 Cookies Results Docs { }

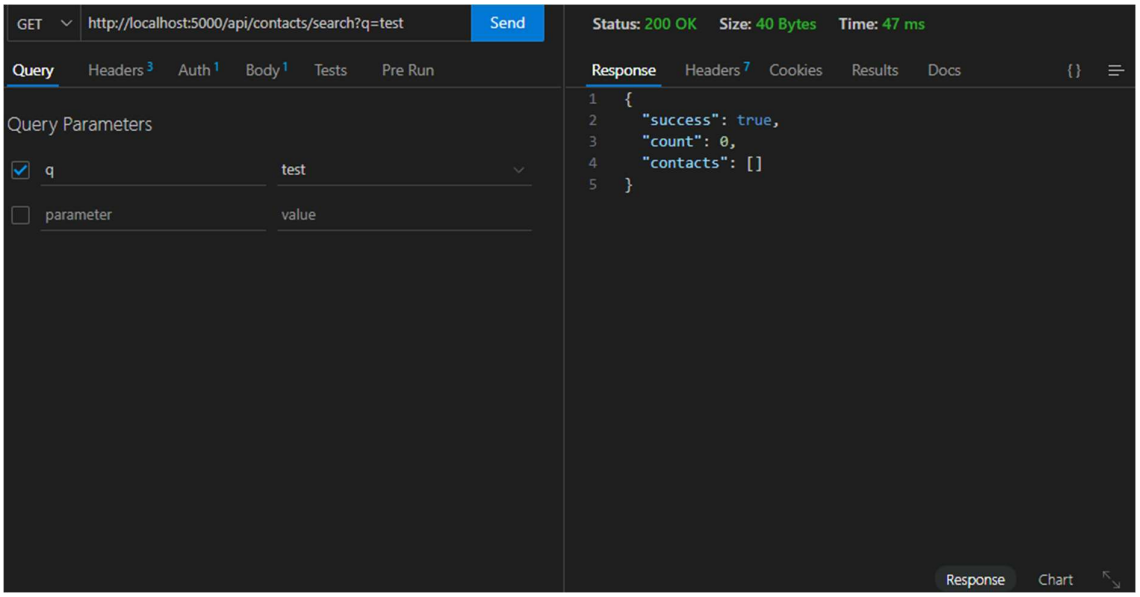
```
{
  "success": true,
  "contact": {
    "_id": "69ab644dfdd29a914e922c62",
    "userId": "69aa93e0c2c4a9e803e2a8d1",
    "name": "sri varthini",
    "email": "srivarthininkrcs@gmail.com",
    "tags": [
      "test"
    ],
    "createdAt": "2026-03-06T23:33:33.505Z",
    "updatedAt": "2026-03-06T23:52:15.561Z",
    "__v": 1,
    "phone": "6547892345"
  }
}
```

Response Chart

6.Search Contact



7.Get The Current User



12. Known Issues:

Slow loading when many contacts exist.

No contact import/export feature.

13. Future Enhancements:

• Import contacts from CSV/Excel

• Mobile application support

• Contact profile picture

• Contact grouping and tagging

• Cloud database integration

Sample Backend Code (Node.js + Express)

Copy code

Javascript

```
const express = require("express");
const app = express();

app.use(express.json());

let contacts = [];

app.get("/contacts", (req, res) => {
   res.json(contacts);
});

app.post("/contacts", (req, res) => {
   const contact = req.body;
   contacts.push(contact);
   res.json({ message: "Contact added successfully" });
});

app.put("/contacts/:id", (req, res) => {
```

```
    const id = req.params.id;
    contacts[id] = req.body;
    res.json({ message: "Contact updated" });
});

app.delete("/contacts/:id", (req, res) => {
    const id = req.params.id;
    contacts.splice(id, 1);
    res.json({ message: "Contact deleted" });
});

app.listen(3000, () => {
    console.log("Server running on port 3000");
});
```